

Broken Promise: Differential Analysis of Functional Discrepancies Between WebAssembly and Native Binaries

Xiao Wu*
xiaowu@hust.edu.cn
Huazhong University of Science and
Technology, Hubei Key Laboratory of
Distributed System Security
Wuhan, China

Alan Romano*
alan.rom@outlook.com
University of Southern California
Los Angeles, United States

Liyan Huang*
liyanhua@usc.edu
University of Southern California
Los Angeles, United States

Qiwen Yan
qiwen@hust.edu.cn
Huazhong University of Science and
Technology, Hubei Key Laboratory of
Distributed System Security
Wuhan, China

Cai Fu[†]
fucui@hust.edu.cn
Huazhong University of Science and
Technology, Hubei Key Laboratory of
Distributed System Security
Wuhan, China

Weihsang Wang[†]
weihsangw@usc.edu
University of Southern California
Los Angeles, United States

Abstract

WebAssembly (Wasm) is a web standard that defines a binary instruction format serving as a portable compilation target for high-level programming languages. As a cornerstone design feature, code porting empowers developers to migrate code across platforms without extensive redevelopment. However, we observe that certain native code exhibits functional discrepancies compared to its Wasm counterpart. These discrepancies impede migration by causing functional alterations, generating misleading messages, and forcing platform-specific workarounds.

In this paper, we present an in-depth analysis of these Native-to-Wasm discrepancies, demonstrating that they often stem from complex underlying causes, leading to unexpected outcomes in real-world applications. We characterize the root causes and practical implications of these discrepancies, providing critical insights into WebAssembly code porting challenges. To facilitate this large-scale analysis, we develop WasmDiff, an automated differential fuzzing framework that detects Native-to-Wasm discrepancies by comparing the execution of identical code across platforms to identify input-triggered behavioral differences. We identify 14,053 discrepancies across 64,001 test samples from the basic benchmark and 3,102 discrepancies within eight real-world projects. Our findings reveal that discrepancies are prevalent, input-dependent and rooted in diverse execution stacks, underscoring the need for greater attention to code porting issues in the WebAssembly ecosystem.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging; Compilers.**

*Equal Contribution.

[†]Both authors serve as corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates.

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792193>

Keywords

WebAssembly; Differential Testing; Cross-compilation

ACM Reference Format:

Xiao Wu, Alan Romano, Liyan Huang, Qiwen Yan, Cai Fu, and Weihsang Wang. 2026. Broken Promise: Differential Analysis of Functional Discrepancies Between WebAssembly and Native Binaries. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792193>

1 Introduction

WebAssembly (Wasm) [37] is a binary instruction format that serves as a compilation target for high-level languages, such as C/C++, Rust, and Go [39]. With the goal of “code once, run everywhere,” Wasm is designed as a portable target that can be deployed to the web and other platforms, such as IoTs [48, 49], smart contracts [28, 61, 65], and cloud computing [41, 53, 62]. With the advent of WebAssembly System Interface (WASI) [29], Wasm can support operating-system-like features such as file systems and networking. Together, these features enable reliable, multi-platform execution from a single source codebase.

While porting native applications to WebAssembly promises significant cost savings over redevelopment, this advantage is often offset by functional discrepancies [60], i.e., ported Wasm produces different outputs for the same inputs. We find that such functional discrepancies are prevalent, manifesting not only in simple benchmarks but also in complex real-world programs. Even simple factors like differences in C libraries can cascade into highly complex discrepancies in large-scale projects. These discrepancies can alter program functionality and disrupt error reporting [18], complicating debugging and impeding practical adoption of Wasm.

While these discrepancies have been noted [14, 16, 17], their broader implications remain underexplored. To the best of our knowledge, no prior work has systematically examined the distribution and impact of discrepancies in real-world applications and under large-scale testing conditions. Existing studies are limited to simple benchmarks [60] or built-in unit tests [23], which

do not adequately reflect the scope of such discrepancies. Given WebAssembly's foundational emphasis on code migration, a more thorough examination of these differences is warranted.

Our work addresses the following key challenges inherent in this analysis: Semantic discrepancies often arise only under specific program inputs, requiring extensive input space exploration (❶). Discrepancies in real-world programs involve complex behavioral patterns, making characterization and impact assessment far more difficult than with basic benchmarks (❷). Undefined Behavior (UB) is permitted by design in language specifications, making it challenging to isolate its effects from legitimate discrepancies in valid code (free from UB) (❸). In this paper, we conduct such an investigation, providing an in-depth analysis of Native-to-Wasm discrepancies revealing their intricate nature and widespread prevalence.

To enable a large-scale analysis, we developed WasmDiff¹, a differential fuzzing framework that compiles identical C/C++ code into native and WebAssembly binaries under same compilation flags, systematically explores inputs likely to cause discrepancies by mutating a seed corpus, and detects discrepancies by comparing program outputs. We implement a prototype of WasmDiff using AFL++ [35]. Through comprehensive evaluation, we identify 14,053 discrepancies across 64,001 Juliet test samples and 3,102 within eight real-world projects. As summarized in Table 1, our analysis reveals key insights regarding the prevalence, triggering conditions, root causes, and practical impact of these discrepancies, underscoring the need for more efforts to improve code porting guarantees in the WebAssembly ecosystem.

In summary, this paper makes the following contributions:

- We conduct a comprehensive analysis of Native-to-Wasm semantic discrepancies, characterizing their root causes, manifestation patterns, and practical implications across both benchmark and real-world codebases.
- We develop WasmDiff, an automated differential fuzzing framework that detects 3.19x more discrepancies than prior manual analysis and identifies eight previously unknown discrepancy subcategories and one new root cause category.
- We identify 14,053 discrepancies from 64,001 test samples in the Juliet benchmark and 3,102 discrepancies from eight portable real-world projects, revealing that discrepancies in real-world programs are more severe – including misleading error messages and critical functional deviations in core program logic.

2 Motivations

2.1 Native-to-Wasm Discrepancy

When porting C/C++ code to WebAssembly, functional inconsistencies can occur that we term *Native-to-Wasm discrepancy*.

DEFINITION 1 (NATIVE-TO-WASM DISCREPANCY). A *Native-to-Wasm discrepancy* occurs when a C/C++ program compiled to both native and Wasm binaries using identical compilation options (e.g., optimization levels, linking configurations, debugging flags) produces different outputs for the same input.

Hereafter, we use *discrepancy* as shorthand. While discrepancies can manifest in intermediate runtime results (e.g., function return values), we focus on output-level differences. Program output is

¹We make WasmDiff publicly available at [9].

MSA Result from Native binary

```
MDYGGALSAVGRELLFVT-----NPVVVNGSVLV-PEDQC
YCAGWIESRGTNGAQTASNVLQW-LAAGFSILLMFYAYQT-W
KSTCGWEEIYVCAIEMVKVILEFFFEKFNPSMLYLATGHRVQWL
RYAEWLLTCP-----VILHLSN-LTGLSNDYSRRTMGLLVSDIGTI
VWGATSAMATGYVKVIFFCGLG-L-CYGANTFFHAAKAYIEGY
HTVPKGRCRQVVTGMAWLFFVSWGM-FPILFILGPEGFVLSVYG
STVGHTIIDLMSKNCWG-LLGHYLRVLIHEHILHGDIRKTTKLNIG
GTEIEVETLVE-DEAEAG-AVNKGTGK
```

MSA Result from Wasm binary

```
MDYGGALSAVGRELLFVTNPVVVNGSVLV-VPED-QCYCAGWIESR
GTNGAQTAS-NVLQW-LAAGFSILLMFYAYQTWKSTCGWEEIYV
CAIEMVKVILEFFFEKFNPS-MLYLATGHRVQWLRYAEWLLTCP-
-----VILHLSNLT--GLSNDYSRRTMGLLVSDIGTIWVGATSAMATG
YVKVIFFCGLG-CYGANTFFHAAKAYIEGYHTVPKGRCRQVVTGWM
AWLFFVSWGM-FPILFILGPEGFVLSVYGSTVGHTIIDLMSKNCW
GLLGHYLRVLIHEHILHGDIRK-TTKLNIGGTEIEVETLVEDEAEAG
AVNKGTGK
```

(a) A capital letter represents an amino acid and '-' represents a blank character inserted in the amino acid sequence.

(b) The red █ represents the whitespace inserted by the native binary in the original sequence, and the yellow █ represents the insertion by the Wasm binary.

Figure 1: Discrepancy Example from Real-world Program (Kalign): Functional Change.

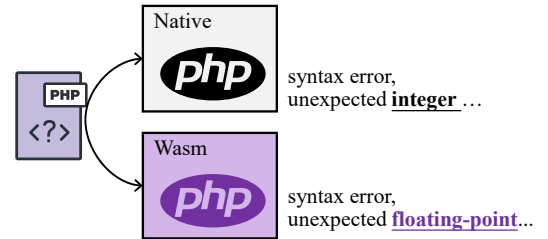


Figure 2: Discrepancy Example from Real-world Program (Php): Confusing Error Message.

the primary indicator of whether functional requirements are satisfied, making output consistency the critical measure for migration success. This scope aligns with prior work [23, 60].

2.2 Basic Discrepancy Example

Figure 4 in Appendix shows a simple Native-Wasm discrepancy from Juliet test suite [24]. The code reads input into an array (Lines 1-2) and checks if fputs succeeds by verifying its return value (Lines 3-5). The native binary (using *glibc* [4]) prints "string", while the Wasm binary (using *musl-libc* [7]) prints "stringfputs failed!". This occurs because *glibc* returns 1 on success, whereas *musl-libc* returns 0, causing the Wasm binary to misinterpret the result.

2.3 Discrepancy Examples in Real-world Programs

Compared to the simple test cases in the basic benchmark, discrepancies in real-world programs usually manifest in more complex behaviors and severe consequences. We illustrate their practical impact through two representative discrepancy examples from real-world programs (Kalign [45] and Libtiff [5]).

Table 1: Key Findings on Native-to-Wasm Discrepancies

	Finding	Implication
1	Prevalence: Discrepancies affect 19.83% of valid code samples (Section 6.1)	Of 32,001 good functions in Juliet, 6,346 exhibited discrepancies, demonstrating that code porting issues persist even in code free of undefined behavior.
2	Scale and Impact: 3,102 discrepancies found across eight real-world projects (Section 6.1), manifesting in complex behaviors (Section 2.3)	Discrepancies in real-world applications compromise WebAssembly’s code porting promise, forcing developers to maintain platform-specific workarounds, exhibiting more severe consequences.
3	Triggering: 53.9% require specific input conditions (Section 6.1)	Most discrepancies manifest under fuzzer-generated inputs, making them difficult to detect under default input conditions.
4	Root causes: Four categories explain all discrepancies (Section 6.2)	Standard library differences, missing security protections, execution environment variations, and compiler differences account for all observed discrepancies.
5	Scope: Systematic analysis reveals 8 new discrepancy subcategories and 1 new root cause category (Section 6.3)	We identified previously unknown categories like libc header differences and memory size mismatches.

(1) *Functional Change.* In bioinformatics, multiple sequence alignment (MSA) tools like Kalign are crucial for analyzing genetic relationships. When porting Kalign to Wasm, we observe divergent alignment results for the same protein sequences (see Figure 1). This discrepancy stems from two key factors: (1) Wasm’s lack of AVX2 [52] instruction support, disabling Kalign’s optimizations, and (2) differing standard library functions (e.g., sorting algorithms) between native and Wasm compilers. While Kalign is heuristic-based, discrepancies in its output indicate substantial functional deviations. Such functional deviations raise concerns about Wasm’s reliability for real-world porting.

(2) *Confusing Error Information.* PHP interpreters of identical versions and compilation parameters may produce divergent diagnostic conclusions when parsing erroneous code. As illustrated in Figure 2, while the native platform’s PHP interpreter identifies an integer-related error, the Wasm counterpart reports a floating-point error for the same code segment. When parsing integers exceeding 32 bits, the PHP interpreter in Wasm defaults to floating-point representation, subsequently generating misleading diagnostic information. Such inconsistent error diagnoses frequently lead to misleading debugging pathways.

The presence of such discrepancies significantly impacts the code migration of Wasm. To this end, we conduct an in-depth investigation of discrepancies in both basic benchmarks and real-world projects, elucidating their distribution, root causes, and implications to aid developer understanding and maintenance.

3 Background

This section introduces WebAssembly’s compilation pipeline and standalone runtimes, explaining our toolchain choices.

3.1 WebAssembly Compilation Pipeline

WebAssembly binaries are usually compiled from high-level languages like C/C++. Emscripten [31] is a popular C/C++-to-Wasm compiler that leverages Binaryen [1], LLVM [6], and Clang [2] to generate Wasm binaries primarily for web environments. While its `STANDALONE_WASM` option enables non-web usage, it supports WASI better on individual files compared to full project compilation. WASI SDK [63] is a WASI-enabled toolchain that focuses on porting complete C/C++ projects to Wasm. Like Emscripten, it uses Clang and

LLVM, but provides a complete C/C++ standard library with WASI support, enabling straightforward compilation of entire projects into WASI-enabled Wasm binaries. We use both WASI SDK and Emscripten in our experiment, as detailed in Section 4.1.

3.2 WebAssembly Standalone Runtime

WASI enables Wasm execution outside browsers. A Wasm runtime decodes the binary into a module, validates it, and executes instructions [13]. The runtime encompasses a "stack" to record operand values and control structures, plus abstract storage containing global state. We use existing standalone runtimes—wasmtime [11], WAVM [12], and WasmEdge [10]—which support direct execution of Wasm binaries in the operating system.

4 WasmDiff

This section introduces WasmDiff, a scalable framework for automatically identifying Native-to-Wasm discrepancies across large codebases. We detail its architecture and implementation, which are designed to enable efficient discrepancy detection at scale. The goal of WasmDiff is to detect discrepancies by comparing the output of the native binary and Wasm binary for the same input. The workflow of WasmDiff includes four components:

- *Compilation*, which uses two compilers C_1 , C_2 to compile the same source code S into native binary B_1 and Wasm binary B_2 using the same compilation options P .
- *Input generation*, which sets an initial corpus of seeds S_{init} for B_1 , B_2 and defines the rules R for generating the inputs. Use code coverage and discrepancy to adjust the input generation strategy.
- *Execution*, which selects the runtime W_1 for the Wasm binary B_2 . Generate inputs according to the rules R , use these inputs to execute B_1 and B_2 , and record their outputs O_1 and O_2 .
- *Discrepancy identification*, which checks if O_1 differs from O_2 or if one binary crashes or hangs during the execution, the corresponding input is recorded.

4.1 Compilation

We use Clang to compile each test sample into an x64 binary. For generating Wasm binaries, we utilize both Emscripten and the WASI SDK. We use the WASI SDK to compile real-world projects into WebAssembly since Emscripten lacks WASI support for real-world

projects [15]. To maintain resemblance to realistic program usage, we opt for using the O2 optimization parameters [60].

4.2 Input Generation

Given the sparse nature of discrepancies in applications, only specific input can trigger them. Unlike using predefined test suites or existing test generation tools, fuzzing leverages a mutation-based approach—a method widely recognized for its practical effectiveness and implementation simplicity. To generate inputs that are more likely to trigger discrepancies, we adapt differential fuzzing algorithms [3, 25, 36] to optimize for discrepancy detection rather than code coverage alone. WasmDiff prioritizes inputs that have previously revealed discrepancies by allocating increased mutation opportunities and computational resources. This approach prioritizes the exploration of uncharted code regions while increasing the likelihood of uncovering new discrepancies. Details on input generation are provided in Appendix B.

4.3 Execution

WasmDiff concurrently executes WebAssembly and native binaries on a Wasm runtime and a native x64 platform, respectively. During execution, it eliminates non-deterministic samples that inherently produce different outputs, e.g., samples containing race conditions, by running each sample 10 times on the same input and platform. To collect code coverage from native binaries, we integrate instrumentation code during compilation. Specifically, we utilize *afl-clang*, the compiler wrapper provided by AFL++, to add necessary instrumentation. For Wasm binaries, we do not introduce any instrumentation for the fast execution.

4.4 Discrepancy Identification

Ideally, identifying discrepancy requires verifying the semantic equivalence of compiled native and Wasm binaries, yet full verification is infeasible due to its inherent complexity and undecidability[32, 54]. In practice, we approximate this by comparing the outputs of native and WebAssembly binaries, following previous work. Following [50], we compare final outputs (stdout, files, errors) for discrepancy identification because (1) they assess functional discrepancy sufficiently; (2) intermediate-state comparisons can cause false positives from compiler optimizations and need non-trivial instrumentation [50]; (3) discrepancies not affecting final results are typically low-priority to developers [50]. This method offers efficiency and simplicity, as output capture incurs minimal overhead compared to complex intrusive instrumentation like sanitizers [50, 64]. Moreover, output-level discrepancies most directly undermine WebAssembly’s code migration goals, making them a critical priority for detection and resolution. When a new discrepancy is discovered or either program crashes, the inputs responsible for triggering the behavior are saved for analysis.

5 Study Design

This section describes our experimental design. We present our evaluation benchmarks (Section 5.1), explain how we analyze root causes (Section 5.2), discuss our handling of undefined behavior (Section 5.3), and detail our experimental setup (Section 5.4).

As shown in Figure 3, we evaluate our approach using a structured workflow that begins with preprocessing basic benchmarks

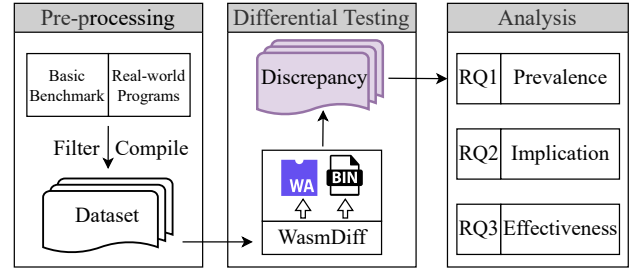


Figure 3: Overview of the experimental procedure.

Table 2: Portable Real-World Projects

Name	Input	Version	Use Case	LOC	Files
Php-cgi [20]	PHP	8.0.0	PHP parsing	1.39M	1
Libtiff [5]	Tiff	4.3.0	Image processing	87.38K	7
Libsndfile [18]	Audio	1.2.2	Audio processing	65.33K	3
Libtomcrypt [19]	Text	1.18.2	Cryptography	74.39K	2
Cmark [30]	Markdown	Latest	Markdown parsing	17.09K	1
Jsmn [68]	JSON	Latest	JSON parsing	931	1
Pdjson [58]	JSON	Latest	JSON parsing	1.43K	1
PDFresurrect [34]	PDF	0.23	PDF analyzing	1.19K	1

and real-world programs, followed by differential testing to uncover discrepancies across compilation outputs. The final analysis addresses the prevalence, implications, and efficiency of these discrepancies through three targeted research questions.

- **RQ1:** What is the prevalence and distribution of Native-to-Wasm discrepancies in benchmark and real-world code?
- **RQ2:** What are the root causes, manifestation, and implications of these discrepancies?
- **RQ3:** Can WasmDiff effectively detect discrepancies, especially at scale in large codebases?

5.1 Evaluation Benchmarks

5.1.1 Juliet test suite. To investigate the prevalence of Native-to-Wasm discrepancies, we employ the Juliet benchmark test suite from NIST [24]. Specifically, we utilize Juliet C/C++ V1.3, which encompasses 118 CWE (Common Weakness Enumeration) types. Each CWE type contains both a correct code sample and a fault-containing code sample, providing diverse test scenarios. In total, this dataset comprises 64,099 test samples across all CWE types.

To ensure reliable analysis, we exclude samples that produce non-deterministic outcomes or cannot be analyzed. Specifically, we exclude the following CWEs: Programs that never terminate (CWE 674, 835); Programs that produce uncertain results (CWE 364, 367, 832); Programs that cannot be compiled to Wasm (CWE 667, 832). After these exclusions, our dataset comprises 64,001 test samples.

5.1.2 Real-World Projects. Table 2 lists eight open-source C/C++ projects that are either highly-starred GitHub repositories or used in published work [42, 43, 47, 50]. These projects represent common WebAssembly use cases such as data processing, media file handling, and cryptography [8]. Since WASI is under active development, most open-source projects include APIs that are incompatible with WASI (e.g., networking, longjmp uses). The eight projects in our study are among the subset that can be successfully compiled to

Wasm using the WASI SDK, making them suitable for investigating discrepancies in realistic porting scenarios.

5.2 Analyzing Root Causes

For each discrepancy, we examined the triggering inputs generated by WasmDiff, traced execution differences between native and Wasm binaries, and inspected compilation artifacts, runtime environments, and library implementations. Our root-cause analysis uses common debugging practices (e.g., inserted prints), while automated inference remains open research. Discrepancies were categorized by root cause based on collaborative analysis and discussion among authors. For real-world projects, we also assessed the practical impact of discrepancies.

5.3 Undefined Behavior

Discrepancies due to undefined behavior (UB) during code migration are expected [50], as UB lies outside programming language specifications and constitutes a legitimate source of discrepancies. However, discrepancies in valid, UB-free code present a more subtle challenge: they frequently evade detection because such code passes conventional test cases on native platforms, creating a false perception of correctness. The Juliet test suite provides patched versions (i.e., valid code) for each CWE. To eliminate interference from undefined behavior, we segregated the suite’s good functions (valid code) from bad functions (flawed code) in equal proportions and conducted separate testing on each category.

5.4 Experimental Setup

We use Clang 10.0.0, Emscripten 3.1.45 (for the Juliet test suite), and WASI SDK 20 (for real-world projects) as the backend compilers for our analysis. We perform our experiments on a server running Ubuntu 20.04 LTS with an Intel(R) Xeon(R) CPU E7-4820 v4 @ 2.00GHz and 128GB RAM.

6 Findings

We systematically analyze discrepancies across the Juliet Test Suite and real-world programs by executing WasmDiff for 120 hours per test case, filtering out duplicate discrepancies. Our key findings are summarized in Table 1.

6.1 RQ1: Prevalence of Discrepancies

6.1.1 Results on the Juliet Test Suite. Our analysis identified 14,053 unique discrepancies of the 64,001 test samples (21.96%) in the test suite, as shown in Table 3. We categorize these discrepancies in Table 4, which will be further detailed in Section 6.2. Notably, 7,577 discrepancies (53.9%) manifested under fuzzer-generated inputs, demonstrating that many discrepancies require specific input conditions to trigger. Critically, discrepancies affected 6,346 of the 32,001 valid code samples (19.83%) rather than bug-containing test cases. This finding reveals that code migration challenges extend well beyond buggy code, significantly impeding WebAssembly’s code porting promise even for well-formed programs.

6.1.2 Results on Real-World Projects. Our analysis identified 3,102 unique discrepancies across eight portable real-world projects, as shown in Table 3, and the categorization is as demonstrated in Table 4. Unlike the Juliet Test Suite, real-world projects exhibit greater complexity. The distribution of discrepancies across the

Table 3: Discrepancies Detected By WasmDiff

Name	Input Type	Samples	Discrepancies
Basic Benchmark			
Juliet Test Suite	None	8,004	1,707
	Default Input	4,911	4,769
	Fuzzer Generated	51,086	7,577
Total		64,001	14,053
Real-World Projects			
Php-cgi	Fuzzer Generated	-	36
Libtiff	Fuzzer Generated	-	2,537
Libsndfile	Fuzzer Generated	-	11
Libtomcrypt	Fuzzer Generated	-	8
Cmark	Fuzzer Generated	-	488
Jsmn	Fuzzer Generated	-	5 (UB)
Pdjson	Fuzzer Generated	-	11
PDFresurrect	Fuzzer Generated	-	6
Total			3,102

projects is notable, with *Libtiff* exhibiting the highest number of discrepancies at 2,537, despite being the second largest project in terms of line of code (87.38K). In contrast, *Php-cgi*, the largest project (1.39M), shows 36 discrepancies. This demonstrates that discrepancy prevalence depends more on code characteristics than on project size alone.

RQ1: Native-to-Wasm discrepancies are pervasive across both benchmarks and real-world code, with distribution varying by code characteristics rather than size in real-world projects. These discrepancies require specific input conditions to manifest and persist even in well-formed code, revealing significant code migration challenges.

6.2 RQ2: Discrepancy Characteristics

To gain a deeper insight into these execution discrepancies, we inspect and classify the root causes of discrepancies from the Juliet test suite and real-world projects into four categories: *Standard Library*, *Security Protections*, *Execution Environment*, and *Compiler*. We exclude undefined behaviors from the results as discussed previously in Section 5.3. Within each category, we pinpoint the specific root cause and provide descriptions in Table 4. Next, we introduce these root causes with code examples.

6.2.1 Standard Library. Wasm binaries are typically compiled using the *musl* [7] library instead of the commonly used *glibc* [4] provided by the OS. If WASI system calls are required, the *wasi-libc* [22] library is also needed, which is a specialized library launched to provide dedicated WASI support. We discover differences in the implementations of these standard libraries that lead to discrepancies in the code after porting. Our analysis finds 10,959 such discrepancies in total.

Wide Characters. We find that discrepancies in string output can arise due to the different wide character mechanisms employed by native and Wasm binaries. We identified 3,349 such discrepancies

Table 4: Categorization of Discrepancies in the Juliet Test Suite and Real-World Projects

Root Cause	Subcategory	Description	Juliet Test Suite	Real-World Project
Standard Library	Wide Characters	<i>glibc</i> sets file stream orientation, unlike <i>musl</i>	3,349	0
	malloc/free puts/put printf/vprintf/nprintf/...	Different function implementation between <i>glibc</i> and <i>musl</i>	413	0
			36	0
			323	0
	rand*	Different pseudo-random number generation method between <i>glibc</i> and <i>musl</i>	5,033	8 (<i>Libtomcrypt</i>)
	File-Related Implementation*	<i>Wasi-libc</i> file permissions differ from <i>glibc</i>	292	0
	Libc Header*	File mode macros in <i>wasi-libc</i> and <i>glibc</i> differ	0	896 (<i>Libtiff</i>)
Security Protections	Clock API*	Default clock API in <i>wasi-libc</i> can only get UTC time while <i>glibc</i> gets current time	0	645 (<i>Libtiff</i>)
	Lack of Stack Protection	Wasm doesn't have stack/heap/page/.. protection	2,118	0
	Lack of Heap Protection*		1,232	0
	Memory Protections		552	0
Execution Environment	Size of Numbers	Pointers/numbers are 32-bit by default in Wasm while 64-bit in native code	18	42 (<i>Php-cgi</i> : 36, <i>PDFresurrect</i> : 6)
	Running Environment	Environment variables will not migrate with code	18	1,007 (<i>Libtiff</i> : 996, <i>Libsndfile</i> : 11)
	Memory Size*	Default memory sizes for Wasm and native differ	39	499 (<i>Pdjson</i> : 11, <i>Cmark</i> : 488)
Compiler*	Optimization results*	Different optimization choice/compilation method	288	0
	Compilation results*		342	0
Total			14,053	3,097

Notes: * indicates new discrepancy categories over PC2W; A real-world project, *jsmn*, is excluded from here because its discrepancies result from uninitialized data, classified as UB.

in the Juliet test suite and 1,549 discrepancies across 3 real-world projects: *Libtiff*, *Php-cgi*, and *Libtomcrypt*. This kind of discrepancy arises from variations in wide-stream implementations in standard C libraries. Figure 5 in Appendix presents code from CWE121 that results in a discrepancy caused by the use of wide characters.

Different Libc Header. Since the C/C++ standard lacks a precise definition for some libc macros, *glibc* and *wasi-libc* utilize different values. These differences in macro definitions lead to divergent control flows within the binaries, resulting in 896 discrepancies identified in the *Libtiff* project. For example, we observe that *wasi-libc* defines file permission macros, such as `O_RDONLY` and `O_RDWR`, differently than *glibc*. Within the *Libtiff* project, these macros are used to indicate the file's current state. Differences in macro-reference lead to variations in the program's logic, resulting in complex bugs. Consequently, for the developer, this error may obscure the true intent of the program's error reporting, thereby complicating the debugging process. Moreover, such discrepancies may potentially lead to security issues as the Wasm binary attempts to allocate memory for problematic inputs. Figure 6 in Appendix shows an example in real-world program *Libtiff*.

Different rand. The `rand()` function generates pseudo-random numbers using a linear congruential generator. To ensure reproducibility of discrepancies, we use `srand()` to seed the generator for consistent results. However, different implementations of `rand()` in *glibc*, *musl-libc*, and *wasi-libc* produce different sequences of random numbers for the same seed. These differences lead to 5,033

discrepancies in the Juliet test suite and 8 discrepancies in the *Libtomcrypt* project. Divergent random number sequences render outcomes of certain test code unpredictable and non-reproducible.

Different File-Related Implementation. *Wasi-libc* does not handle filesystems in the same way as native libraries. As such, some functions, for example, `fopen`, fail to create files, which causes discrepancies in 292 cases. The Wasm binary does not throw errors on file operation failures, consequently invalidating the program's original logic for checking file existence, size, and permissions.

6.2.2 Security Protections. Unlike native platforms, Wasm lacks stack or heap protection mechanisms. Although the WebAssembly documentation suggests that the absence of these mechanisms does not compromise security [21], we find that their absence can result in discrepancies when handling problematic code. Our analysis identifies 3,902 such discrepancies.

Lack of Heap Protections. We discover 1,232 discrepancies attributable to the absence of heap protection in WebAssembly. As a result, illegal operations such as heap smashing, which typically result in crashes in x64, can execute successfully in Wasm.

Lack of Stack Protections. In native binaries, certain stack smashing operations, such as buffer overflow, can be prevented by stack protection measures, e.g., stack canaries. However, since WebAssembly lacks these stack protection mechanisms, WebAssembly will allow the execution of illegal stack operations. We find 2,118 such discrepancies.

6.2.3 Execution Environment. We encounter discrepancies that are caused by differences in the runtime environments between native and Wasm binaries. We find differences in number sizes, environment variables, and memory size can all potentially cause changes in the program’s functionality. Our analysis detects 1,118 such discrepancies.

Different Size of Numbers. Since Wasm defaults to 32-bit applications, some 64-bit integers could be recognized as 32-bit numbers within the binary. Different sizes of numbers can cause deviations in their actual values in the Wasm binary, leading to discrepancies. Our analysis finds 18 discrepancies in the Juliet test suite and 42 discrepancies in projects *Php-cgi* and *PDFresurrect*. This seemingly minor issue produces highly counterintuitive results in Figure 2 and significantly increases debugging complexity for maintainers.

Different Running Environment. During compilation, compilers link useful common libraries implicitly. These libraries can define macros to indicate that support exists for specific functionalities. For example, the `OJPEG_SUPPORT` macro is used in the *Libtiff* and *Libsndfile* projects to determine whether the current environment supports JPEG for compression operations. However, this macro is undefined in the Wasm runtime, leading to different control flow when reading input. We find 18 discrepancies in the Juliet test suite and a total of 1,007 discrepancies in *Libtiff* and *Libsndfile*.

Memory Size. By default, WebAssembly modules are allocated a default size of linear memory (256 MB) that can be resized as needed. This initial memory size is smaller than memory available to native binaries, leading to issues. For example, if a large memory address is accessed, such as 4294967300 (512 MB), in a native binary, this number is still within the normal range of values that a program can request, so no error is reported. However, in a Wasm binary, the default memory range in WasmEdge is 256 MB, so the instructions to access this address generate out-of-bounds exceptions in Wasm. We identify 39 such discrepancies in the Juliet test suite and 499 discrepancies in real-world projects *Pdjson* and *Cmark*.

6.2.4 Compiler 630 discrepancies are caused by differences occurring during compilation of the source code to native and Wasm binaries. Importantly, these issues are not caused by compiler bugs. Rather, the different compilation settings are both correct in isolation but lead to differences when compared together. We divide this root cause into two categories: *Different optimization results*, and *Different compilation results*.

Different Optimization Results. We find that, even at the same optimization level, native and Wasm compilers can apply different optimizations to the same code, which may alter the functionality of the generated program. This root cause leads to 288 discrepancies in the Juliet test suite.

Different Compilation Results. We find 342 discrepancies in the Juliet test suite are caused by the Wasm compiler incorrectly generating binary instructions. Figure 7 in Appendix shows an example from CWE194.

While basic benchmarks demonstrate simple discrepancy patterns, real-world applications reveal more complex behavioral divergences that often compromise fundamental program operations. Our findings reveal that real-world discrepancies frequently manifest in two critical forms: alterations to core program functionality (see in Figure 1), or misleading error responses to malformed inputs

Table 5: Comparing Discrepancies Found in PC2W against WasmDiff

Root cause	Due to	PC2W	WasmDiff
Standard library	Wide characters	3,253	3,349
	malloc/free	259	413
	puts	36	36
	printf	26	323
Security protections	Stack smashing	626	2,118
	Memory protections	143	552
Execution environment	Uninitialized data	382	1,537
	Size of pointers	26	-
	Size of numbers	18	18
	OS environment	18	18
	Memory layout	18	-

(see in Figure 2 and Figure 6). Both scenarios significantly escalate maintenance and code-review burdens post-migration. Even for experienced developers, diagnosing such issues within large, complex codebases remains nontrivial. These challenges fundamentally undermine WebAssembly’s migration guarantees.

RQ2: Our analysis identifies four main types of discrepancies caused by: different standard libraries, missing security protections, execution environment variations, and compiler. Compared to basic benchmark cases, discrepancies in real-world programs exhibit greater complexity and can critically impact core functionality.

6.3 RQ3: Scalable Effectiveness of WasmDiff

6.3.1 Basic Benchmark. We compare our results against the manual work performed in PC2W [60], which analyzed the causes of discrepancies only on a basic benchmark, Juliet Test Suite. Table 5 presents the categories and counts obtained in PC2W as well as the discrepancy counts obtained by WasmDiff. Designed for large-scale, complex code spaces, WasmDiff nonetheless shows stronger performance on basic, smaller benchmarks compared to manual analysis.

Overall, we find 14,053 unique discrepancies in the Juliet test suite, a 3.19x increase compared to the 4,911 discrepancies found in PC2W. We document all the discrepancies identified by PC2W as well as 11,864 new ones. Specifically, we identify eight new subcategories of discrepancy root causes and 1 new root cause category (Table 4), *Compiler*. The increased number and diversity of discrepancies identified by WasmDiff, as opposed to the manual analysis via PC2W, can be attributed to the employment of coverage-based differential fuzzing. This technique not only automates the input generation, leading to enhanced code coverage but also expedites the program’s execution and the output comparison.

WasmDiff checks only final outputs and does not record intermediate states (e.g., return values or memory), so it misses pointer-size discrepancies that appear as divergent return values and memory-layout discrepancies that depend on intermediate memory. Recording intermediate states is a priority for future work.

6.3.2 Real-world Projects. WasmDiff detected 3,097 discrepancies in eight real-world projects. Some discrepancies identified by WasmDiff are noticeable via manual investigation (*Different Clock API*, etc.). However, other discrepancies (e.g., *Different Libc Header*) are hard to detect in regular program use as they require specific flawed inputs. These types typically involve complex logical conditions and data dependencies, as a result, the position of the root cause can be far from the location where the output differences occur. Additionally, the error reason presented by the Wasm binary often differs from the actual cause. WasmDiff, as an automated detection tool using differential-fuzzing, can help developers easily and efficiently detect these discrepancies generated during C/C++ code porting and lower the cost of adopting Wasm.

RQ3: WasmDiff identifies 3.19x more discrepancies than manual analysis, uncovering one new root cause category and eight novel subcategories, while its automated detection makes large-scale analysis of real-world projects more feasible. Overall, WasmDiff demonstrates substantially improved effectiveness over pure manual analysis.

7 Discussion

7.1 Code Porting and Discrepancies

In cross-compilation scenarios, discrepancies often arise from code migration and are typically triggered by undefined behavior [27, 33, 50]. However, WebAssembly’s foundational design goal as the code porting target, combined with the discrepancies (e.g., observed in invalid code handling), highlights particularly critical security implications that demand attention. Our analysis finds that different standard libraries, the lack of security protections in Wasm, varying runtime environments, and compilers can all lead to discrepancies. These discrepancies may appear minor on simple tests, but in large, complex code spaces, they can fundamentally alter behavior, cause erroneous outputs, and produce misleading error reports that confuse developers. This would significantly increase the cost and maintenance burden of porting code to Wasm by requiring Wasm-specific compilation adjustments. Resolving this challenge requires coordinated efforts across the WebAssembly community, including toolchain developers and project maintainers, due to the unique combination of host environment variations and language-specific semantics. Through our detailed analysis of existing discrepancies, we aim to draw broader attention to these critical issues.

7.2 Threats to Validity

Internal Validity. The primary internal threats involve configuration and environment bias. First, different optimization levels may yield distinct discrepancy patterns but we adopt the O2 only. Second, the evaluation is limited to Ubuntu and three specific Wasm runtimes, potentially affecting the observed discrepancy distribution across other operating systems or engines.

External Validity. The generalizability of our findings may be constrained by the use of the Juliet test suite’s relatively small synthetic cases. To mitigate this, we evaluated WasmDiff on eight diverse real-world projects, successfully identifying discrepancies and confirming the practical relevance of our results. Our study focuses on Wasm’s C/C++ toolchain and standalone runtimes, since about

64.2% of the real-world Wasm binaries originate from C/C++ [39]. However, WasmDiff’s language-agnostic design allows our testing and analysis to be generalized with minimal modification.

Construct Validity. Fuzzing comprehensiveness poses a construct threat. The absence of a universal framework may lead to oversimplified or biased fuzz targets, impacting efficiency. Furthermore, WasmDiff focuses on observable output differences; it may miss “silent” discrepancies that do not manifest in the output.

8 Related Work

WebAssembly Analysis. Stiévenart et al. [60] manually analyze discrepancies between native and Wasm binaries on the Juliet benchmark test suite, though their manual approach limits the scale and diversity of discrepancies detected. Baradaran et al. [23] investigate challenges in cross-compilation to WebAssembly but rely on codebases’ built-in unit tests, which provide limited scope of potential discrepancies. Unlike these approaches, WasmDiff employs automated differential fuzzing to systematically explore input spaces and detect discrepancies that manifest only under specific conditions—cases that are impractical to identify through manual analysis or basic test suites.

Other works have explored WebAssembly from dynamic analysis and symbolic execution frameworks [38, 46], empirical studies on real-world Wasm usage [39], compiler bug analysis [55], side-channel security [40], functional purpose identification [56], and inlining optimizations [57]. In contrast, our work targets functional discrepancies that arise during Wasm code cross-compilation, which is an unexplored area.

Differential Fuzzing. Some works employ differential fuzzing approaches to identify output discrepancies in programs. Differential fuzzing has also been used to compare Go and C/C++ libraries [59], detect unstable code [50], detect performance issues server-side Wasm [44], optimization problems of the Wasm compiler [51], and Wasm runtime bugs [26, 66]. Unlike these works, WasmDiff focuses on detecting cross-platform functional discrepancies of WebAssembly during code porting rather than runtime or compiler bugs.

9 Conclusion

This study analyzes Native-to-Wasm functional discrepancies, revealing their prevalence in benchmarks and real-world projects. We show that systematic variations in libraries, security mechanisms, and environments cause widespread inconsistencies in valid code, compromising execution integrity. We introduce WasmDiff, an automated differential fuzzing framework that outperforms manual detection. Our results demonstrate that these discrepancies impair program functionality and produce misleading outputs, highlighting a critical need for the community and developers to resolve systemic misalignments to achieve seamless cross-platform execution.

Acknowledgements

We thank the anonymous reviewers for their constructive feedback. This research was supported in part by the U.S. National Science Foundation under grants 2409005 and 2321444. Any opinions, findings, and conclusions in this paper are those of the authors only and do not necessarily reflect the views of our sponsors.

References

- [1] [n. d.]. Binaryen. <http://webassembly.github.io/binaryen/>.
- [2] [n. d.]. Clang C Language Family Frontend for LLVM. <https://clang.llvm.org/>.
- [3] [n. d.]. Full-Speed Fuzzing: Reducing Fuzzing Overhead through Coverage-Guided Tracing | IEEE Conference Publication | IEEE Xplore. <https://ieeexplore.ieee.org/document/8835316>
- [4] [n. d.]. The GNU C Library. <https://www.gnu.org/software/libc/>.
- [5] libtiff [n. d.]. libtiff. <https://gitlab.com/libtiff/libtiff>
- [6] [n. d.]. The LLVM Compiler Infrastructure Project. <https://llvm.org/>.
- [7] [n. d.]. The Musl C Library. <https://musl.libc.org/>.
- [8] [n. d.]. Use Cases - WebAssembly. <https://webassembly.org/docs/use-cases/>
- [9] [n. d.]. WasmDiff. <https://github.com/WebAssembly-Diff/wasmdiffs>.
- [10] [n. d.]. WasmEdge. <https://github.com/WasmEdge/WasmEdge/>.
- [11] [n. d.]. wasmtime. <https://github.com/bytecodealliance/wasmtime/>.
- [12] [n. d.]. WAVM. <https://github.com/WAVM/WAVM/>.
- [13] [n. d.]. WebAssembly specification. <https://webassembly.org/>.
- [14] 2021. Discrepancies problems in instantiateWasm. <https://github.com/emscripten-core/emscripten/issues/14777>.
- [15] 2023. An Emscripten Issue. <https://github.com/emscripten-core/emscripten/pull/1248>.
- [16] 2025. Discrepancy between dart analyze and dart compile js wasm. <https://github.com/dart-lang/sdk/issues/60235>.
- [17] 2025. Inconsistent Pixel Data Between WASM and Binary Executable. <https://github.com/ffmpegwasm/ffmpeg.wasm/issues/818>.
- [18] 2025. Libsndfile. <https://github.com/libsndfile/libsndfile>.
- [19] 2025. libtomcrypt. <https://github.com/libtom/libtomcrypt>.
- [20] 2025. php-cgi. <https://github.com/docker-library/php>.
- [21] 2025. The security of WebAssembly. <https://webassembly.org/docs/security/>.
- [22] 2025. WASI libc implementation for WebAssembly. <https://github.com/WebAssembly/wasi-libc/>.
- [23] Sara Baradaran, Liyan Huang, Mukund Raghothaman, and Weihang Wang. 2024. Reusing Legacy Code in WebAssembly: Key Challenges of Cross-Compilation and Code Semantics Preservation. *arXiv:2412.20258 [cs.SE]* <https://arxiv.org/abs/2412.20258>
- [24] Paul Black. 2018. Juliet 1.3 Test Suite: Changes From 1.2. doi:10.6028/NIST.TN.1995
- [25] Marcel Böhm, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. 2017. Directed greybox fuzzing. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*.
- [26] Shangdong Cao, Ningyu He, Xinyu She, Yixuan Zhang, Mu Zhang, and Haoyu Wang. 2023. WRTTester: Differential Testing of WebAssembly Runtimes via Semantic-aware Binary Generation. (2023). doi:10.48550/ARXIV.2312.10456
- [27] Junjie Chen, Jibesh Patra, Michael Pradel, Yingfei Xiong, Hongyu Zhang, Dan Hao, and Lu Zhang. 2020. A survey of compiler testing. *ACM Computing Surveys (CSUR)* 53, 1 (2020), 1–36.
- [28] Weimin Chen, Zihan Sun, Haoyu Wang, Xipao Luo, Haipeng Cai, and Lei Wu. 2022. WASAI: uncovering vulnerabilities in Wasm smart contracts. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. 703–715.
- [29] Lin Clark. 2019. Standardizing WASI: A system interface to run WebAssembly outside the web. *Mozilla Hacks—the Web developer blog* (2019).
- [30] commonmark. [n. d.]. cmark. <https://github.com/commonmark/cmark>.
- [31] Emscripten Contributors. [n. d.]. *Main — Emscripten 3.1.55-Git (Dev) Documentation*. <https://emscripten.org/>
- [32] Mila Dalla Preda, Roberto Giacobazzi, Arun Lakhotia, and Isabella Mastrosini. 2015. Abstract symbolic automata: Mixed syntactic/semantic similarity analysis of executables. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 329–341.
- [33] Vijay D'Silva, Mathias Payer, and Dawn Song. 2015. The correctness-security gap in compiler optimization. In *2015 IEEE Security and Privacy Workshops*. IEEE, 73–87.
- [34] enfere. [n. d.]. pdfresurrect. <https://github.com/enfere/pdfresurrect>.
- [35] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. 2020. AFL++: Combining Incremental Steps of Fuzzing Research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association. <https://www.usenix.org/conference/woot20/presentation/fioraldi>
- [36] Harrison Green and Thanassis Avgerinos. 2022. GraphFuzz: library API fuzzing with lifetime-aware dataflow graphs. In *Proceedings of the 44th International Conference on Software Engineering*.
- [37] Andreas Haas, Andreas Rossberg, Derek L Schuff, Ben L Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. 2017. Bringing the web up to speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*.
- [38] Ningyu He, Zhehao Zhao, Jikai Wang, Yubin Hu, Shengjian Guo, Haoyu Wang, Guangtai Liang, Ding Li, Xiangqun Chen, and Yao Guo. 2023. Eunomia: Enabling User-specified Fine-Grained Search in Symbolically Executing WebAssembly Binaries. *arXiv preprint arXiv:2304.07204*.
- [39] Aaron Hilbig, Daniel Lehmann, and Michael Pradel. 2021. An empirical study of real-world webassembly binaries: Security, languages, use cases. In *Proceedings of the web conference 2021*. 2696–2708.
- [40] Liyan Huang, Junzhou He, Chao Wang, and Weihang Wang. 2025. WaSCR: A WebAssembly Instruction-Timing Side Channel Repairer. In *Proceedings of the ACM on Web Conference 2025* (Sydney NSW, Australia) (WWW '25). Association for Computing Machinery, New York, NY, USA, 4562–4571. doi:10.1145/3696410.3714693
- [41] Wenjun Huang and Marcus Paradies. 2021. An evaluation of webassembly and ebpf as offloading mechanisms in the context of computational storage. *arXiv preprint arXiv:2111.01947* (2021).
- [42] Joonun Jang and Huy Kang Kim. 2019. FuzzBuilder: Automated building grey-box fuzzing environment for C/C++ library. In *Proceedings of the 35th Annual Computer Security Applications Conference*.
- [43] Bokdeuk Jeong, Joonun Jang, Hayoon Yi, Jiin Moon, Junsik Kim, Intae Jeon, Taesoo Kim, WooChul Shim, and Yong Ho Hwang. 2023. UTOPIA: automatic generation of fuzz driver using unit tests. In *2023 IEEE Symposium on Security and Privacy (SP)*.
- [44] Shuyao Jiang, Ruiying Zeng, Zihao Rao, Jiazhen Gu, Yangfan Zhou, and Michael R Lyu. 2023. Revealing Performance Issues in Server-side WebAssembly Runtimes via Differential Testing. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*.
- [45] Timo Lassmann and Erik LL Sonhammer. 2005. Kalign—an accurate and fast multiple sequence alignment algorithm. *BMC bioinformatics* 6, 1 (2005), 1–9.
- [46] Daniel Lehmann and Michael Pradel. 2019. Wasabi: A framework for dynamically analyzing webassembly. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*.
- [47] Daniel Lehmann, Martin Toldam Torp, and Michael Pradel. 2021. Fuzzm: Finding Memory Bugs through Binary-Only Instrumentation and Fuzzing of WebAssembly. *arXiv:2110.15433 [cs.CR]* <https://arxiv.org/abs/2110.15433>
- [48] Borui Li, Wei Dong, and Yi Gao. 2021. Wipro: A webassembly-based approach to integrated iot programming. In *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*. IEEE, 1–10.
- [49] Borui Li, Hongchang Fan, Yi Gao, and Wei Dong. 2022. Bringing webassembly to resource-constrained iot devices for seamless device-cloud integration. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services*. 261–272.
- [50] Shaohua Li and Zhendong Su. 2023. Finding Unstable Code via Compiler-Driven Differential Testing. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 238–251.
- [51] Zhibo Liu, Dongwei Xiao, Zongjie Li, Shuai Wang, and Wei Meng. 2023. Exploring Missed Optimizations in WebAssembly Optimizers. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 436–448.
- [52] Wojciech Mula, Nathan Kurz, and Daniel Lemire. 2018. Faster population counts using AVX2 instructions. *Comput. J.* 61, 1 (2018), 111–120.
- [53] Mohammed Nurul-Hoque and Khaled A Harras. 2021. Nomad: Cross-Platform Computational Offloading and Migration in Femtoclouds Using WebAssembly. In *2021 IEEE International Conference on Cloud Engineering (IC2E)*. IEEE, 168–178.
- [54] XiaoLei Ren, Michael Ho, Jiang Ming, Yu Lei, and Li Li. 2021. Unleashing the hidden power of compiler optimization on binary code difference: an empirical study. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 142–157.
- [55] Alan Romano, Xinyue Liu, Yonghui Kwon, and Weihang Wang. 2021. An empirical study of bugs in webassembly compilers. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 42–54.
- [56] Alan Romano and Weihang Wang. 2023. Automated WebAssembly Function Purpose Identification With Semantics-Aware Analysis. In *Proceedings of the ACM Web Conference 2023* (Austin, TX, USA) (WWW '23). Association for Computing Machinery, New York, NY, USA, 2885–2894. doi:10.1145/3543507.3583235
- [57] Alan Romano and Weihang Wang. 2023. When Function Inlining Meets WebAssembly: Counterintuitive Impacts on Runtime Performance. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*.
- [58] skeeto. [n. d.]. pdjson. <https://github.com/skeeto/pdjson>.
- [59] Alessandro Sorniotti, Michael Weissbacher, and Anil Kurmus. 2023. Go or No Go: Differential Fuzzing of Native and C Libraries. In *IEEE Symposium on Security and Privacy*.
- [60] Quentin Stiévenart, Coen De Roover, and Mohammad Ghafari. 2022. Security risks of porting c programs to WebAssembly. In *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing*. 1713–1722.
- [61] Dong Wang, Bo Jiang, and WK Chan. 2020. WANA: Symbolic execution of wasm bytecode for cross-platform smart contract vulnerability detection. *arXiv preprint arXiv:2007.15510* (2020).
- [62] wasmCloud Project Authors. 2022. WasmCloud. <https://wasmcloud.com/>.
- [63] WebAssembly Community Group. 2025. WASI-enabled WebAssembly C/C++ toolchain. <https://github.com/WebAssembly/wasi-sdk>. Accessed: July 19, 2025.

- [64] Xiao Wu, Junzhou He, Liyan Huang, Cai Fu, and Weihang Wang. 2025. WB-San: WebAssembly Bug Detection for Sanitization and Binary-Only Fuzzing. In *Proceedings of the ACM on Web Conference 2025*. 3311–3322.
- [65] Shuyu Zheng, Haoyu Wang, Lei Wu, Gang Huang, and Xuanzhe Liu. 2020. VM matters: a comparison of WASM VMS and EVMS in the performance of blockchain smart contracts. *arXiv preprint arXiv:2012.01032* (2020).
- [66] Shiyao Zhou, Muhui Jiang, Weimin Chen, Hao Zhou, Haoyu Wang, and Xiapu Luo. 2023. WADIFF: A Differential Testing Framework for WebAssembly Runtimes. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. <https://api.semanticscholar.org/CorpusID:265054525>
- [67] Xiaogang Zhu, Sheng Wen, Seyit Camtepe, and Yang Xiang. 2022. Fuzzing: a survey for roadmap. *ACM Computing Surveys (CSUR)* 54, 11s (2022), 1–36.
- [68] zserge. [n. d.]. jsnm. <https://github.com/zserge/jsnm>.

Appendices

A Code Examples for Discrepancies

```
1 char input[10] = { 0 };
2 scanf("%s",input); //if input "string"
3 if (fputs(input, stdout) == 0){
4     printf("%s\n", "fputs failed!");
5 }
6 //Native output: string
7 //Wasm output: stringfputs failed!
```

Figure 4: Discrepancy Example from Juliet Test Suite

```
1 #define CHAR_STRING "
2 #define WIDE_STRING L"
3 void *data;
4 data = (void *)CHAR_STRING;
5 /* POTENTIAL FLAW: treating pointer as a char* when it
6    may point to a wide string*/
7 size_t dataLen = strlen((char *)data);
8 void *dest1 = (void *)alloca((dataLen + 1) * 1);
9 void *dest2 = (void *)alloca((dataLen + 1) * sizeof(
10    wchar_t));
11 (void)strcpy(dest1, data);
12 (void)wscpy(dest2, data);
13 if (dest1 != NULL && dest2 != NULL){
14     printf("%s\n", (char *)dest1);
15     wprintf(L"%ls\n", (char *)dest2);
16 }
```

Figure 5: Wide Character Example from CWE121

Figure 5 presents code from CWE121 that results in a discrepancy caused by the use of wide characters. When we compare the outputs between the x64 and Wasm binaries, we find that the native binary prints CHAR_STRING containing 43 “A” characters once (Line 1), whereas the Wasm binary prints both CHAR_STRING and WIDE_STRING (Line 2). According to the C standard, each FILE stream has an associated orientation, set to either “byte” or “wide” mode upon the first operation. When the printf call on Line 12 successfully outputs, the FILE stream (standard output console) is set to “byte” mode. Consequently, wprintf, expecting “wide” mode, fails in x64. While the Clang package on Ubuntu uses the glibc library

[5], Emscripten employs the musl [8] library for its standard C library. Unlike glibc, Musl does not enforce orientation conversion, allowing wide characters to follow regular characters directly. This discrepancy arises from variations in wide-stream implementations between the two standard C libraries. Compiling the x64 binary using musl-gcc [9] resolves this discrepancy.

```
1 int m = -1;
2 switch (mode[0]){
3     case 'r':
4         m = O_RDONLY; // Glibc: 0x00, Wasi-libc: 0
5         if (mode[1] == '+')
6             m = O_RDWR; // Glibc: 0x02, Wasi-libc: 0
7         break;
8     case 'w':
9     case 'a':
10        m = O_RDWR | O_CREAT // Glibc: 0x40, Wasi-libc: 0
11        if (mode[0] == 'w')
12            m |= O_TRUNC; // Glibc: 0x200, Wasi-libc: 0x8000
13        ...
14 }
```

Figure 6: Libtiff Code Getting the Status of the Input File.

In Figure 6, the code for determining the input file’s state uses these differing macro definitions in the variable m. Given a faulty .tiff file, the binary should report an error and exit the program after reading the file. However, since the macro used in the comparison is different within the Wasm binary, the program fails to exit. Instead, it attempts to allocate memory for the file and outputs a completely different error message. This difference in macro values between the libraries causes the *Libtiff* project’s original error diagnosis code to inaccurately deduce the program’s error cause.

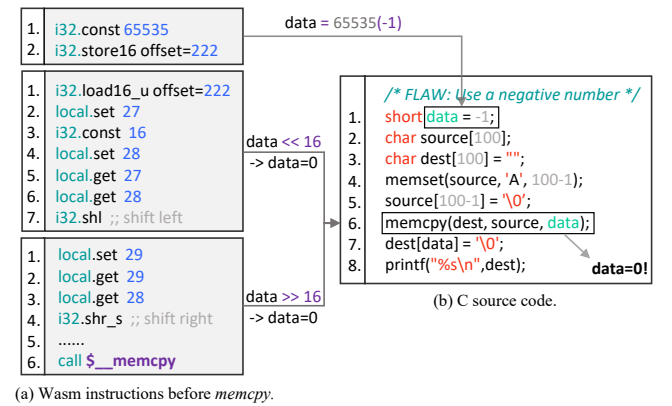


Figure 7: Different Compilation Results Example from CWE194

We present an example from CWE 194 where negative numbers are used for sign extension in Figure 7. The parameter data passes to memcpy has a value of -1 (Line 6). Since memcpy expects an unsigned

integer, the value is interpreted as 65535. This change causes an overflow of the array `dest` as it is only 100 bytes in length. In the native binary, the program crashes, but the Wasm binary ends and prints nothing. We can see how the variable data changes at the instruction level before being passed into the `memcpy`. When used as the length argument in the `memcpy` call, Emscripten compiles `-1` as the maximum unsigned short value 65535. Later, the value 65535 is shifted left by 16, and then shifted right by 16, resulting in the value 0. This is because, in the case of unsigned integers, when we perform a left shift or right shift on a number, the excess bits are discarded. As a result, no action is performed, and no exception occurs.

B Detailed Process of Input Generation

Seed Selection. To prioritize the discovery of inputs triggering discrepancies, we assign mutators more likely to do so with higher fitness during testing. Fitness is a scalar value that quantifies the quality of a seed input:

$$Score(S_i) = D_i \sum_{b \in B(S_i)} \frac{w_b}{n_b}$$

Here, S represents the seed corpus, S_i represents the i -th seed, $Score(S_i)$ represents the actual fitness score of a seed, and D_i represents the discrepancy coefficient. Seeds detecting more discrepancy instances have higher D_i values. $B(S_i)$ indicates the set of basic

blocks covered by S_i , with w_b as the weight of basic block b and n_b as the number of seeds covering basic block b . This fitness function allows seeds that trigger more discrepancies and achieve higher coverage more time to run.

Seed Scheduling. The efficacy of fuzzing directly impacts the efficiency of software exploration [67]. The duration allocated to running a seed is defined as follows in the formula, where $x(S_i)$ represents the length of the seed:

$$time(S_i) = \frac{Score^{1/4}(S_i)}{1 + \log|x(S_i)|}$$

Seeds with higher fitness, shorter length (for taking a balance with time), and higher D_i values are more likely to be allocated a longer runtime.

Mutation Scheduling. We maintain the application probability of each mutator, denoted as $P(M_j)$, and update it after each run, where M represents the set of mutators, and M_j represents the j -th mutator. We employ the existing mutator from current fuzzing work [25].

$$P(M_j) = \frac{D_j \cdot Score(M_j)}{\sum_{k \in M} Score(M_k)}$$

Following each test round, we update the $P(M_j)$ of all mutators to ensure that the mutation operations effectively explore the program space for discrepancies as much as possible.